

UNIVERSIDAD CATÓLICA BOLIVIANA
« SAN PABLO »

Carrera de Ingeniería de Sistemas



INFORME FINAL · PROYECTO INTEGRADOR

Predicción de Precios Inmobiliarios

*Sistema de Tasación de Viviendas con Aprendizaje Automático
sobre el dataset Ames Housing*

Asignatura	SIS-351 — Machine Learning
Metodología	CRISP-DM
Grupo	Grupo 4
Dataset	Ames Housing — OpenML ID 41211
Docente	Ing. Ovidio Roger Patón Gutiérrez

INTEGRANTES

Jean Marco Fernández Silva
Sergio Alejandro Arias Mayta
Marvin Larry Mollo Ramírez
Jaime Ignacio Huaycho Clavel
Sergio Alexander Mendoza Choque

ÍNDICE

Resumen Ejecutivo	1
1 Business Understanding (Fase CRISP-DM 1)	1
1.1 Definición del Problema	1
1.2 Variable Objetivo y Criterios de Éxito	1
1.3 Restricciones del Proyecto	2
2 Data Understanding — EDA (Fase CRISP-DM 2)	2
2.1 Ficha Técnica del Dataset	2
2.2 Variables Clave y Correlaciones	2
2.3 Riesgos de Datos Identificados	4
3 Data Preparation (Fase CRISP-DM 3)	5
3.1 Limpieza y Tratamiento de Outliers	5
3.2 Mapeo de Variables Ordinales	5
3.3 Feature Engineering — 7 Variables Nuevas	5
3.4 Transformaciones del Pipeline	5
3.5 Partición Estratificada 70 / 15 / 15	6
3.6 Pipeline Reproducible	6
4 Modeling (Fase CRISP-DM 4)	6
4.1 Mini Proyecto 1 — Machine Learning Clásico	6
4.2 Mini Proyecto 2 — Componente No Supervisado (K-Means)	7
4.3 Mini Proyecto 2 — Deep Learning	8
4.4 Mini Proyecto 3 — MLOps + RAG (Modelado del Despliegue)	9
5 Evaluation — Resultados (Fase CRISP-DM 5)	9
5.1 Resultados Comparativos — Test Set (440 propiedades)	9
5.2 Validación Cruzada — 5-Fold CV en Train	10
5.3 Importancia de Variables (Random Forest y XGBoost)	11
5.4 Análisis de Error e Interpretabilidad (SHAP)	11
5.5 Análisis de Error por Segmento de Precio	13
5.6 Conclusión Técnica: ¿Cuándo Conviene Deep Learning?	13
6 Deployment — MLOps + RAG (Fase CRISP-DM 6)	13
6.1 Tracking de Experimentos con MLflow	13
6.2 Recuperación Semántica — RAG Tabular con FAISS	14
6.3 Monitoreo y Política de Reentrenamiento	15
6.4 Demostración Funcional	16
7 Integración Final y Continuidad Metodológica	16
8 Conclusiones, Limitaciones y Trabajo Futuro	16
8.1 Conclusiones	16
8.2 Limitaciones	17
8.3 Trabajo Futuro	17

9 Reflexión Crítica y Ética	17
9.1 Reflexión Crítica Metodológica	17
9.2 Sesgos y Equidad	17
9.3 Transparencia e Interpretabilidad	18
9.4 Uso Responsable	18
9.5 Privacidad de Datos	18
9.6 Honestidad Académica	18
10 Referencias	18
11 Anexos	19
11.1 Anexo A — Estructura del Repositorio	19
11.2 Anexo B — Configuración del AutoML (FLAML)	20
11.3 Anexo C — Instrucciones de Ejecución	20
11.4 Anexo D — Reproducibilidad	20

RESUMEN EJECUTIVO

Este informe documenta el desarrollo integral de un **sistema de tasación de viviendas** sobre el dataset **Ames Housing** (2.930 propiedades, 81 variables), siguiendo la metodología **CRISP-DM** de extremo a extremo. La solución se construye de forma progresiva mediante tres mini proyectos articulados sobre un mismo dominio: Machine Learning clásico, Deep Learning y MLOps + recuperación semántica (RAG tabular).

En **Machine Learning** se entrenaron y compararon tres modelos de regresión (Ridge, Random Forest, XGBoost) junto a un **benchmark obligatorio con AutoML (FLAML)**. El modelo **Ridge** obtuvo el mejor desempeño en el conjunto de prueba con **$R^2 = 0,9359$** y **RMSE = \$20.035**, superando incluso a la búsqueda automática. El componente **no supervisado** (K-Means, $K = 4$) identificó cuatro segmentos de mercado interpretables. En **Deep Learning** se implementó un **MLP tabular** y un **Autoencoder** no supervisado (latente 32D), confirmando que en datos tabulares pequeños los modelos clásicos superan a las redes profundas, aunque los **embeddings** aprendidos aportan valor real en la recuperación de similares. En **MLOps + RAG** se construyó un flujo reproducible con **MLflow**, recuperación con **FAISS** (Precisión@5 = 70,4 %), un **dashboard de monitoreo por segmento** y un sistema de **detección de deriva (PSI)**.

- **Resultado principal:** el modelo Ridge alcanza **$R^2 = 0,9359$** y **RMSE = \$20.035** sobre el conjunto de prueba, con un pipeline completamente reproducible y un ciclo MLOps operativo de extremo a extremo.

1 BUSINESS UNDERSTANDING (FASE CRISP-DM 1)

1.1 Definición del Problema

El dataset **Ames Housing** contiene información de 2.930 ventas residenciales en Ames, Iowa (EE. UU.) entre 2006 y 2010 — periodo que incluye la crisis hipotecaria de 2008. La pregunta central es: ¿es posible construir un **sistema automatizado de tasación** que prediga el precio de venta de una propiedad de forma **precisa, interpretable y reproducible**, usando únicamente sus características estructurales y de localización, y que además ofrezca **propiedades comparables** que contextualicen la estimación?

1.2 Variable Objetivo y Criterios de Éxito

La variable objetivo es `Sale_Price` (USD), continua con distribución asimétrica positiva (skewness = 1,744). Los criterios de éxito definidos y los resultados obtenidos:

Dimensión	Métrica / Criterio	Meta mínima	Resultado obtenido
Precisión predictiva	R^2 en test set	$\geq 0,85$	0,9359 ✓
Error en USD	RMSE / MAE	Reportado en USD	\$20.035 / \$13.398 ✓
Interpretabilidad	Top variables	Explicadas	SHAP aplicado ✓

Dimensión	Métrica / Criterio	Meta mínima	Resultado obtenido
Reproducibilidad	Pipeline ejecutable	README requirements +	Pipeline sklearn ✓
Benchmark AutoML	Comparación justa	Misma partición	FLAML 300s ✓
Recuperación (RAG)	Precisión@5	≥ 60 %	70,4 % ✓
Operación (MLOps)	Tracking + monitoreo	Mínimo viable	MLflow + PSI ✓

1.3 Restricciones del Proyecto

- **Recursos computacionales:** entorno local o Google Colab, sin GPU dedicada.
- **Sin Transformers:** el dataset es puramente tabular; conforme a la política oficial de la asignatura, no se exige Transformer salvo columna textual justificada.
- **Dataset estático:** sin actualización en tiempo real de los datos fuente.
- **Alcance geográfico:** limitado a Ames, Iowa, EE. UU. (2006–2010).
- **Sesgo geográfico potencial:** el precio correlaciona con el barrio (ver Sección 9).

2 DATA UNDERSTANDING — EDA (FASE CRISP-DM 2)

2.1 Ficha Técnica del Dataset

El dataset fue publicado por Dean De Cock (2011) en el *Journal of Statistics Education* como alternativa moderna al Boston Housing. Se descargó desde **OpenML (ID 41211)**, con trazabilidad registrada en `data/processed/metadata.json`.

Atributo	Valor
Registros totales	2.930 propiedades vendidas
Variables totales	81 (80 predictoras + Sale_Price)
Variables numéricas	35 (superficies, conteos, años, valoraciones)
Variables categóricas	46 (zonas, tipos, calidades, vecindarios)
Precio mínimo / máximo	\$12.789 — \$755.000
Precio mediano / promedio	\$160.000 / \$180.801
Skewness del precio	1,744 (asimetría positiva)
Periodo temporal	2006 – 2010 (incluye crisis 2008)
Valores faltantes	4.599 celdas

2.2 Variables Clave y Correlaciones

Las variables más correlacionadas con Sale_Price, coherentes con el dominio inmobiliario:

Variable	Corr. (r)	Descripción
Gr_Liv_Area	0,707	Superficie habitable sobre suelo (ft²)
Garage_Cars	0,648	Capacidad del garaje (N.º autos)
Garage_Area	0,640	Área física del garaje (ft²)
Total_Bsmt_SF	0,633	Superficie total del sótano (ft²)

Variable	Corr. (r)	Descripción
First_Flr_SF	0,622	Superficie del primer piso (ft²)
Year_Built	0,558	Año de construcción original
Full_Bath	0,546	Número de baños completos
Year_Remod_Add	0,533	Año de la última remodelación

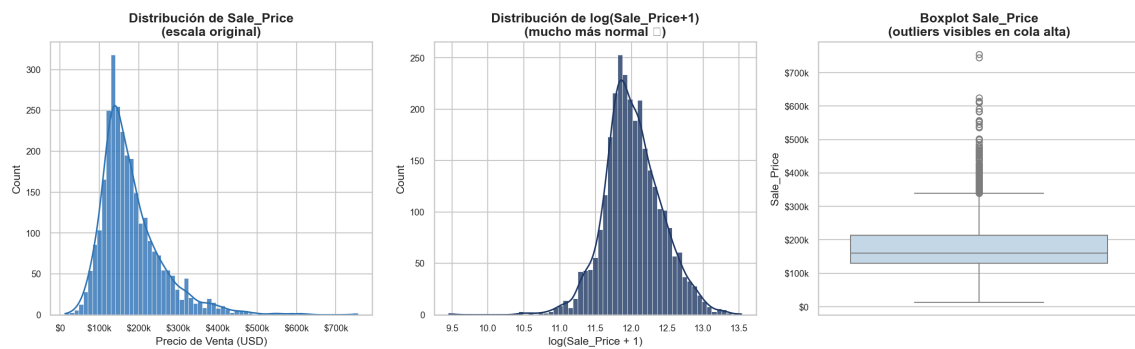


Figura 1: Distribución de Sale_Price antes y después de la transformación logarítmica. La transformación normaliza la cola derecha (skew = 1,744) y estabiliza la varianza.

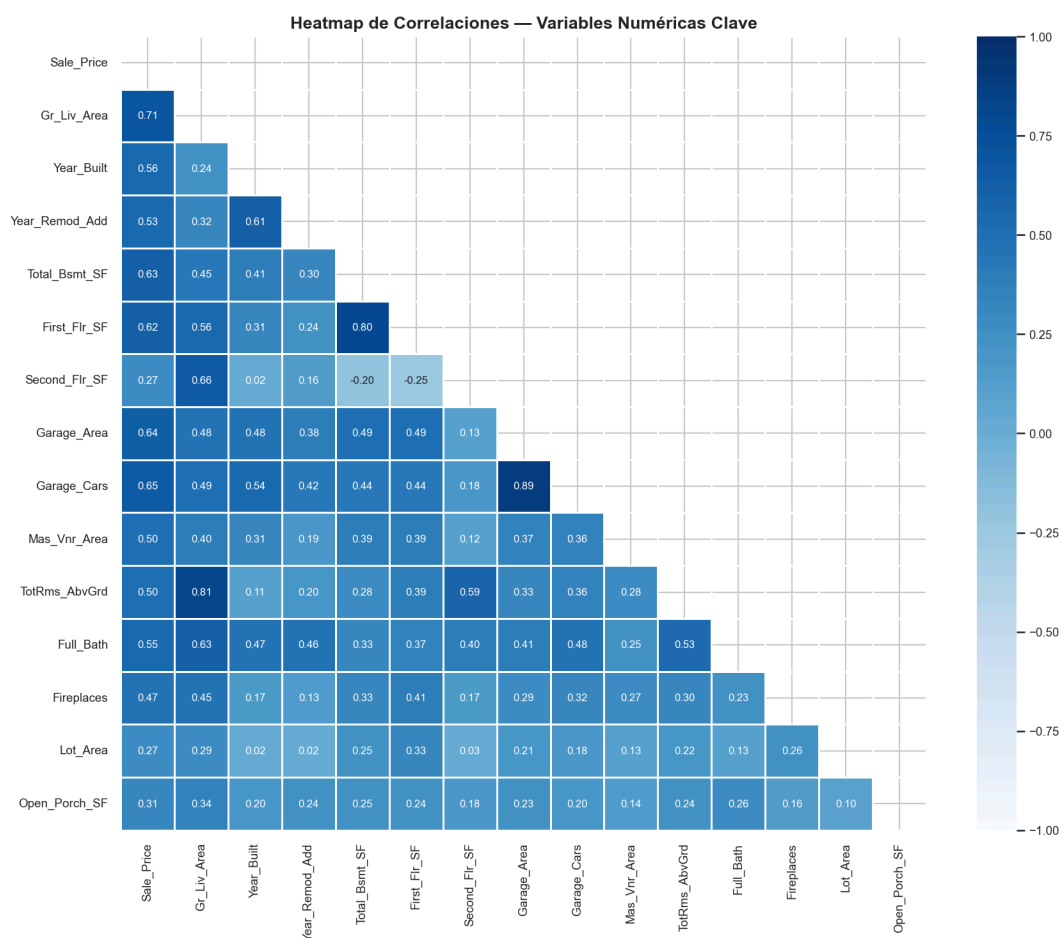


Figura 2: Mapa de calor de correlaciones entre las principales variables numéricas.

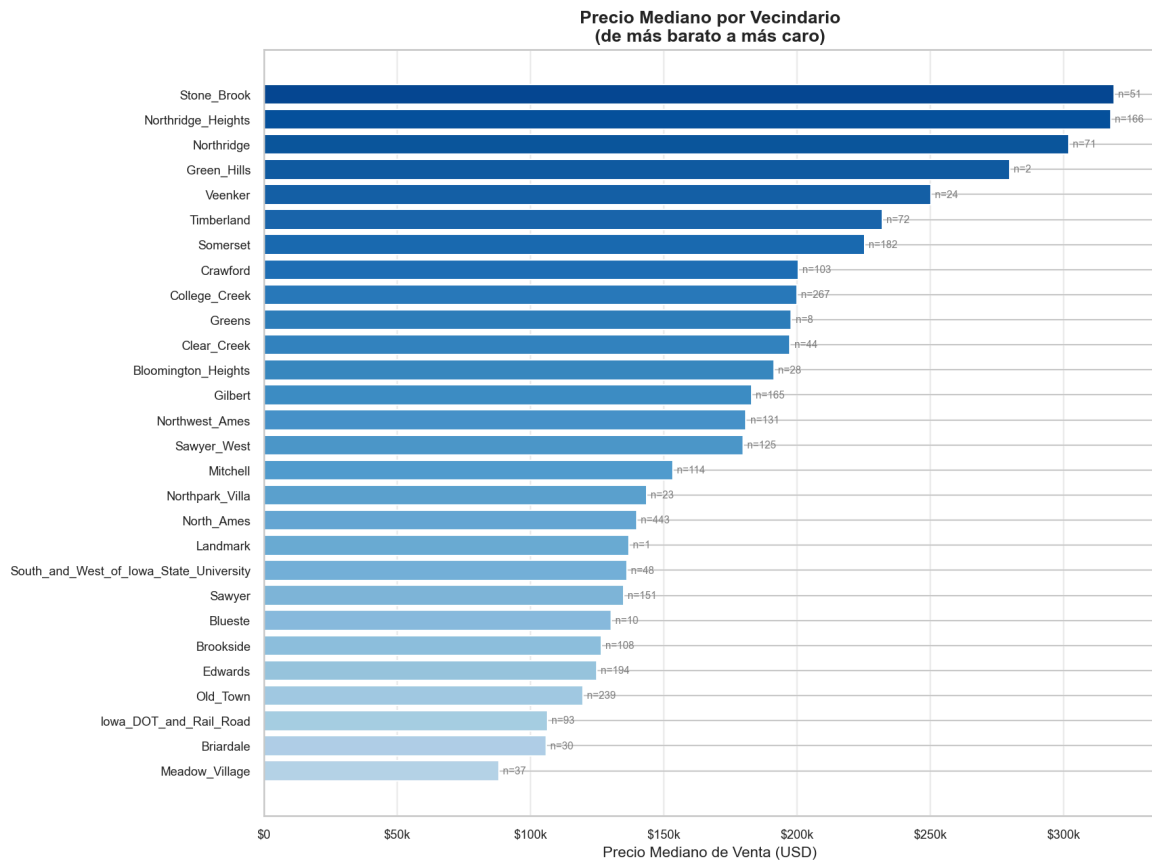


Figura 3: Precio mediano por barrio. La fuerte variación entre vecindarios evidencia el peso de la ubicación y motiva la reflexión ética sobre sesgo geográfico.

2.3 Riesgos de Datos Identificados

- **Valores faltantes estructurales:** Misc_Feature (2.824) y Mas_Vnr_Type (1.775) faltan porque la característica no existe en la vivienda (ausencia real, no error). Estrategia: imputación por mediana / categoría «None».
- **Outliers documentados:** 3 propiedades con Gr_Liv_Area > 4.000 ft² y precio bajo, ventas atípicas señaladas por De Cock (2011). Excluidas del entrenamiento.
- **Multicolinealidad:** Garage_Area ↔ Garage_Cars, Gr_Liv_Area ↔ TotRms_AbvGrd. Afecta a modelos lineales (mitigado con regularización L2).
- **Distribución asimétrica del objetivo:** solución aplicada $\log_{10}()$ en el target, revertido con $\exp_{10}()$ al reportar en USD.
- **Sin leakage ni duplicados** relevantes tras la inspección.

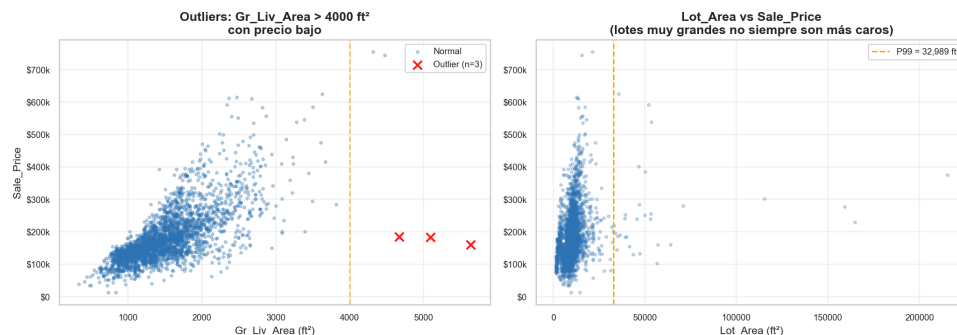


Figura 4: Detección de outliers en Gr_Liv_Area vs Sale_Price.

3 DATA PREPARATION (FASE CRISP-DM 3)

3.1 Limpieza y Tratamiento de Outliers

Se eliminaron las **3 propiedades atípicas** con $Gr_Liv_Area > 4.000 \text{ ft}^2$ y precio bajo (ventas atípicas documentadas por De Cock, 2011), reduciendo el dataset de **2.930 a 2.927 registros**.

3.2 Mapeo de Variables Ordinales

19 columnas ordinales con valores textuales se mapearon a una escala numérica preservando su orden semántico (p. ej. Poor = 1, Fair = 2, TA = 3, Good = 4, Excellent = 5; escala 1–10 para Overall_Qual y Overall_Cond). Esto evita que el One-Hot Encoding pierda la relación de orden de estas variables.

3.3 Feature Engineering — 7 Variables Nuevas

Se crearon siete variables derivadas que capturan relaciones que los modelos no aprenden fácilmente de las variables individuales. La más relevante, **Total_SF**, alcanzó $r = 0,832$ con Sale_Price, **superando a todas las variables originales** y convirtiéndose en el predictor más importante del proyecto.

Variable nueva	Definición	Relevancia
Total_SF	Total_Bsmt_SF + First_Flr_SF + Second_Flr_SF	r = 0,832 ★ máxima
Total_Baths	Full + 0,5×Half (interior y sótano)	Alta correlación
House_Age	Year_Sold – Year_Built	Negativa (antigüedad)
Remod_Age	Year_Sold – Year_Remod_Add	Negativa (renovación)
Was_Remodeled	1 si Year_Built ≠ Year_Remod_Add	Indicadora binaria
Total_Porch_SF	Suma de todos los tipos de porche	Correlación media
Has_Pool	1 si Pool_Area > 0	Diferenciador premium

3.4 Transformaciones del Pipeline

Tipo de variable	Tratamiento aplicado
Objetivo (Sale_Price)	Transformación log1p (corrige skew = 1,744)
Numéricas con skew > 1	log1p sobre Gr_Liv_Area, Lot_Area, Total_Bsmt_SF, First_Flr_SF
Numéricas (faltantes)	SimpleImputer (mediana) → StandardScaler
Categorías (faltantes)	SimpleImputer (constante «missing»)
Categorías	One-Hot Encoding (handle_unknown = ignore)

El ColumnTransformer procesa **60 columnas numéricas** (imputadas y escaladas) y **27 categorías** (imputadas y codificadas con OHE). Tras el encoding, el espacio pasa de 81 columnas crudas a **263 features numéricas** listas para el modelado.

3.5 Partición Estratificada 70 / 15 / 15

La partición es **estratificada por quintiles de precio** (`random_state = 42`). Toda la preparación se **ajusta solo en train** y se aplica a `val` y `test`, eliminando data leakage. Las medianas prácticamente idénticas validan la estrategia:

Conjunto	Proporción	Registros	Features
Entrenamiento (Train)	70,0 %	2.048	263
Validación (Val)	15,0 %	439	263
Prueba (Test)	15,0 %	440	263

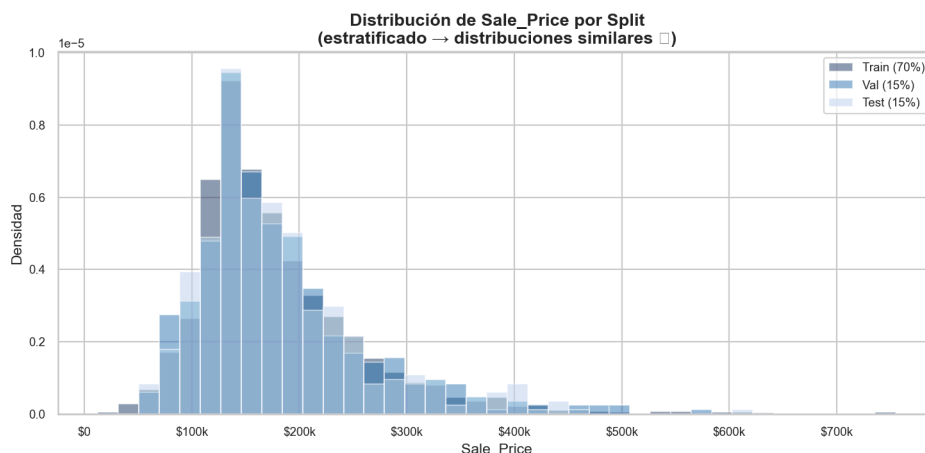


Figura 5: Distribución de precios consistente entre train/val/test gracias a la estratificación por quintiles.

3.6 Pipeline Reproducible

Todo el preprocesamiento está encapsulado en `src/preprocessing.py` (`ColumnTransformer`) y parametrizado en `config/params.yaml`. La trazabilidad se garantiza con hashes MD5 de cada artefacto:

— **Trazabilidad (metadata.json):** `X_train.npy` → `a7dc7f403a8b` · `X_test.npy` → `533b03c23ecb` · `preprocessor.pkl` → `f425b9b337b4`. Cualquier cambio en los datos altera el hash y es detectable.

4 MODELING (FASE CRISP-DM 4)

4.1 Mini Proyecto 1 — Machine Learning Clásico

Se entrenaron tres modelos comparables —un **baseline lineal explícito** (Ridge) y modelos no lineales (Random Forest, XGBoost)— sobre las mismas 263 features y la misma partición.

Modelo	Tipo	Decisión clave
Ridge (baseline)	Lineal L2	RidgeCV seleccionó $\alpha = 10,0$; más simple y explicable
Random Forest	Bagging	300 árboles; robusto a outliers, sin escalado

Modelo	Tipo	Decisión clave
XGBoost	Boosting	500 estimadores, lr = 0,05; estado del arte tabular

4.1.1 Benchmark Obligatorio con AutoML (FLAML)

Se ejecutó **FLAML** con presupuesto de 300 s sobre la **misma partición y protocolo**. FLAML seleccionó internamente un `xgb_limitdepth` (1.291 estimadores, profundidad 2, lr = 0,059).

— **Lectura crítica del AutoML:** FLAML alcanzó $R^2 = 0,9311$, resultado **muy competitivo pero inferior** al Ridge manual ($R^2 = 0,9359$). Para este problema tabular, un modelo lineal simple, interpretable y barato **igual o supera** a la búsqueda automática, que entrega un modelo más complejo y opaco. AutoML aquí **valida** la elección manual; no la sustituye.

4.2 Mini Proyecto 2 — Componente No Supervisado (K-Means)

Se aplicó K-Means sobre 9 variables semánticamente ricas (incluyendo las nuevas `Total_SF`, `Total_Baths` y `House_Age`). El número **K = 4** se seleccionó con el **método del codo (Elbow)** y el **coeficiente de silueta** (Silhouette = 0,240 para K = 4; inercia = 10.921). La proyección PCA 2D explica el 77,5 % de la varianza (PC1 = 62,2 %, PC2 = 15,3 %). Se identificaron cuatro segmentos de mercado interpretables (medianas):

Cluster	Tamaño	Precio mediano	Calidad	Año	Segmento
0	30,7 %	\$127.000	5,0	1960	Económico
1	36,8 %	\$185.000	7,0	1997	Premium (mayoría)
2	16,0 %	\$300.000	8,0	2004	Lujo
3	16,5 %	\$133.000	5,0	1923	Gama media / histórico

Los clusters tienen **coherencia geográfica**: los vecindarios premium (StoneBr, NridgHt) concentran propiedades del cluster Lujo, mientras que los más humildes (MeadowV, BrDale) caen en el segmento Económico.

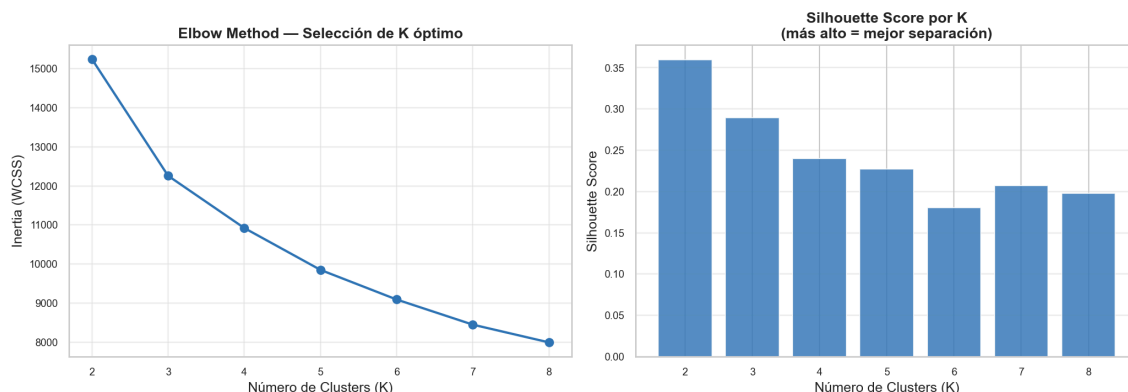


Figura 6: Selección de K mediante método del codo y coeficiente de silueta.

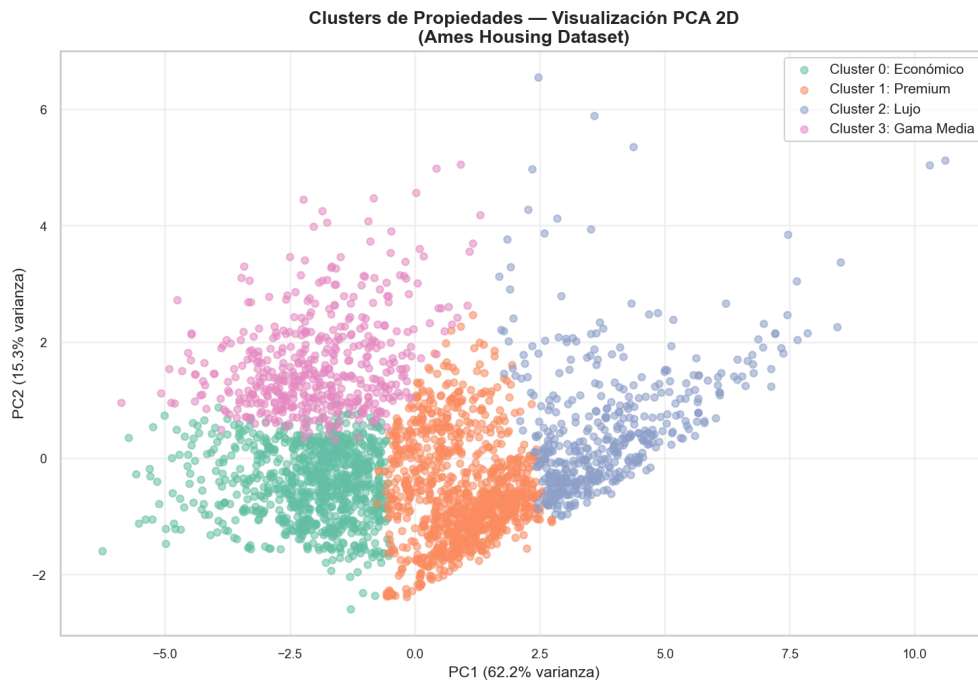


Figura 7: Proyección PCA 2D de los 4 clusters de propiedades.

4.3 Mini Proyecto 2 — Deep Learning

4.3.1 MLP Tabular

Se implementó un Perceptrón Multicapa en PyTorch con arquitectura $263 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 1$ (301.313 parámetros). Cada bloque combina Linear $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Dropout(0,3). El entrenamiento usó Adam, ReduceLR0nPlateau y **Early Stopping**, deteniéndose en la **época 113**.

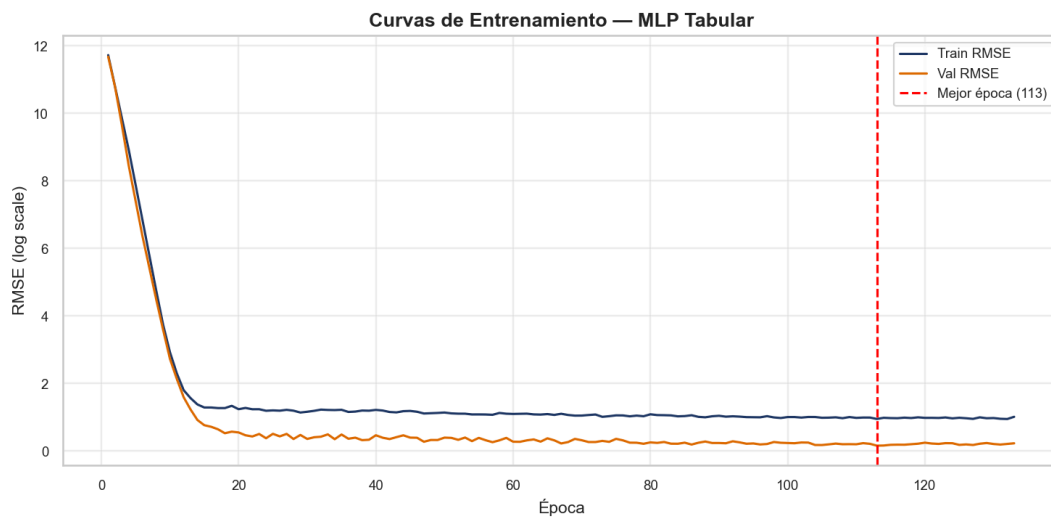


Figura 8: Curvas de entrenamiento y validación (RMSE log). El early stopping en la época 113 evita el sobreajuste: la validación deja de mejorar mientras la de entrenamiento seguiría descendiendo.

4.3.2 Autoencoder Tabular (no supervisado)

Como segundo enfoque profundo se entrenó un **Autoencoder** $263 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 263$ (89.319 parámetros), optimizando la reconstrucción (MSE) **sin usar el precio** (aprendizaje no supervisado). Su latente de **32D** comprime el espacio original a solo el **12,2 %** de sus dimensiones, ofreciendo una representación compacta alternativa para la búsqueda de similitud.

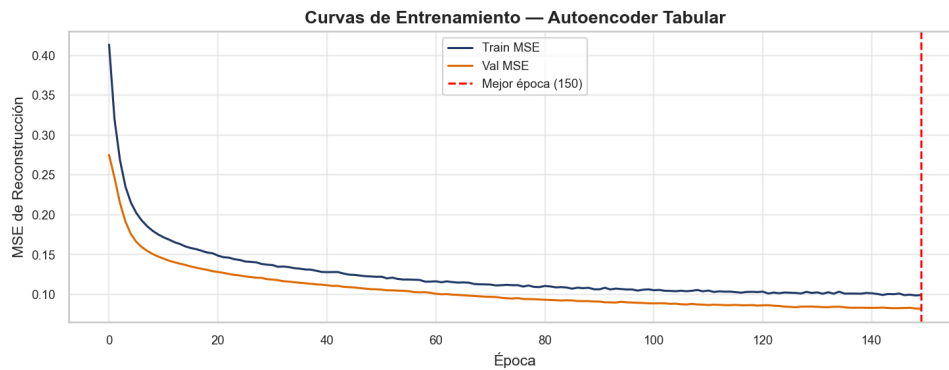


Figura 9: Curva de pérdida de reconstrucción (MSE) del Autoencoder.

4.4 Mini Proyecto 3 — MLOps + RAG (Modelado del Despliegue)

Componente MLOps	Herramienta / Implementación
Tracking de experimentos	MLflow: hiperparámetros, métricas y artefactos por run
Versionado de datos	Registro en <code>metadata.json</code> con hashes MD5
Versionado de modelos	MLflow Model Registry: Staging → Production
Pipeline de inferencia	<code>src/predict.py</code> reproducible con modelo versionado
Recuperación (RAG)	FAISS IndexFlatIP sobre embeddings 128D del MLP
Monitoreo propuesto	Dashboard de error por segmento + PSI
Reentrenamiento	Trigger por deriva (PSI $\geq 0,25$)

5 EVALUATION — RESULTADOS (FASE CRISP-DM 5)

5.1 Resultados Comparativos — Test Set (440 propiedades)

Todos los modelos se evalúan sobre el **mismo conjunto de prueba**, con las **mismas features** y **mismas métricas**. El target fue `log1p(Sale_Price)`; las métricas se reportan en USD.

Modelo	R^2	RMSE (USD)	MAE (USD)	RMSE log
Ridge — Baseline ★	0,9359	\$20.035	\$13.398	0,1164
AutoML FLAML	0,9311	\$20.773	\$13.552	0,1219
XGBoost	0,9219	\$22.106	\$13.576	0,1159
Random Forest	0,9017	\$24.805	\$14.972	0,1260
MLP Tabular (DL)	0,8619	\$29.401	\$20.388	0,1463

Todos los modelos superaron $R^2 \geq 0,85$. **Ridge Regression**, siendo el más simple, obtuvo el mejor **R^2 en test (0,9359)**, superando a XGBoost, Random Forest, al MLP profundo e incluso a AutoML, sin justificar la pérdida de interpretabilidad de los modelos más complejos.

Comparación de Modelos — Test Set

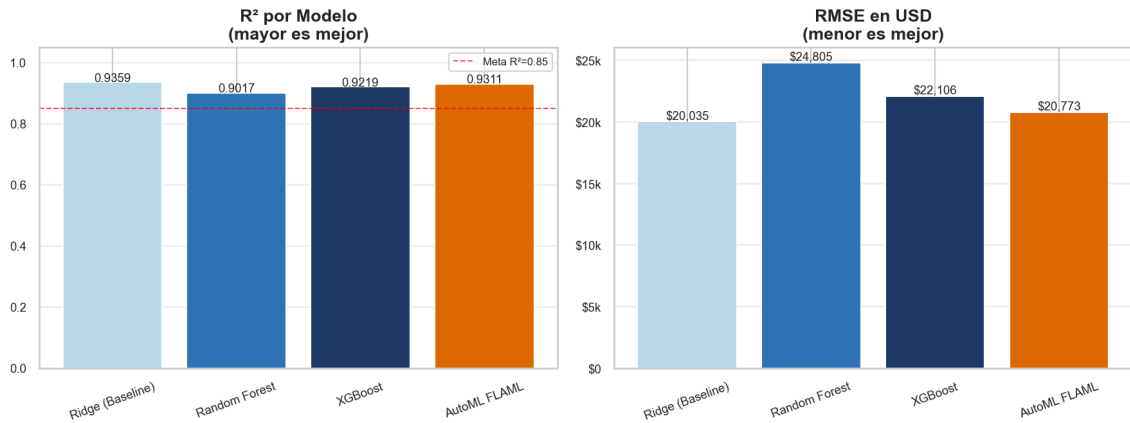


Figura 10: Comparación visual de R² y RMSE entre todos los modelos.

ML Clásico vs Deep Learning — Test Set



Figura 11: Deep Learning vs. Machine Learning clásico. El MLP no supera a los modelos clásicos en este dataset tabular pequeño.

5.2 Validación Cruzada — 5-Fold CV en Train

Para verificar la **estabilidad** de los modelos más allá de una sola partición, se ejecutó una validación cruzada de 5 folds sobre el conjunto de entrenamiento. Ridge no solo es el más preciso en test, sino también el **más estable** (menor RMSE medio):

Modelo	RMSE log (media)	Desv. estándar
Ridge — Baseline ★	0,1161	± 0,0155
XGBoost	0,1263	± 0,0143
Random Forest	0,1367	± 0,0133

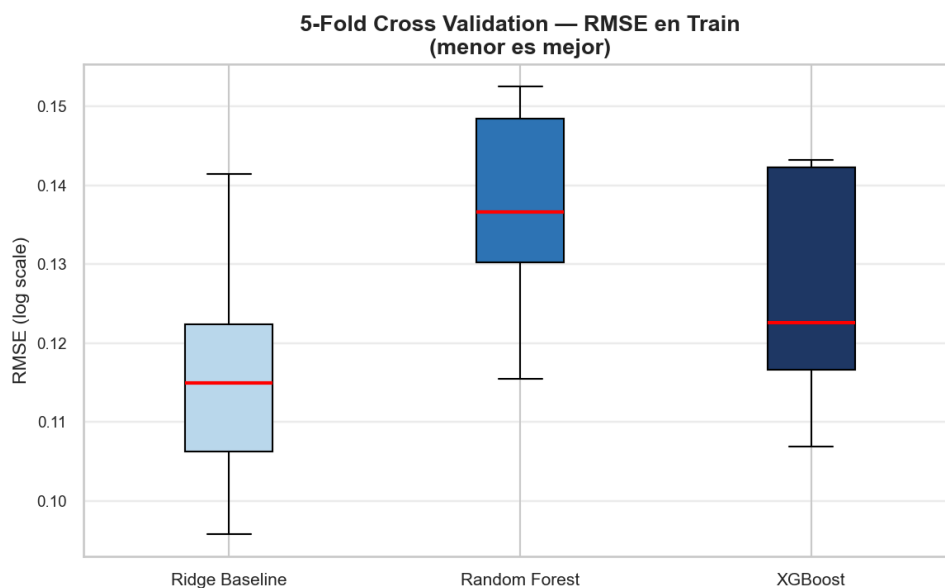


Figura 12: Distribución del RMSE en los 5 folds de validación cruzada por modelo.

5.3 Importancia de Variables (Random Forest y XGBoost)

Los modelos de árboles coinciden en las variables más determinantes: **Overall_Qual** y **Total_SF** aparecen en el top de ambos, confirmando su robustez como predictores.

Random Forest	Imp.	XGBoost (top SHAP)	Imp.
Overall_Qual	0,479	Overall_Qual	0,126
Total_SF	0,309	Total_SF	0,124
Total_Baths	0,018	Lot_Area	0,020
Gr_Liv_Area	0,014	Gr_Liv_Area	0,017
Garage_Area	0,013	Total_Baths	0,016

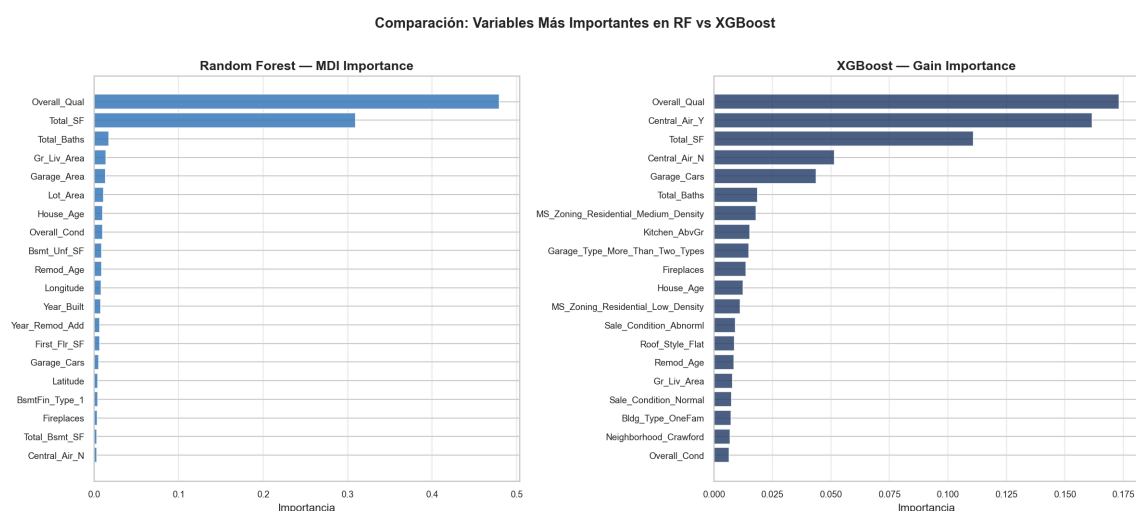


Figura 13: Comparación de importancia de variables entre Random Forest y XGBoost.

5.4 Análisis de Error e Interpretabilidad (SHAP)

No basta con el puntaje global: se aplicó **SHAP** (TreeExplainer) sobre el modelo **XGBoost** para explicar cada predicción individual. El análisis confirma que **Overall_Qual** (0,126) y **Total_SF** (0,124)

son, con diferencia, los factores más determinantes del precio, seguidos de Lot_Area, Gr_Liv_Area y Total_Baths. El valor base del modelo es \$165.861.

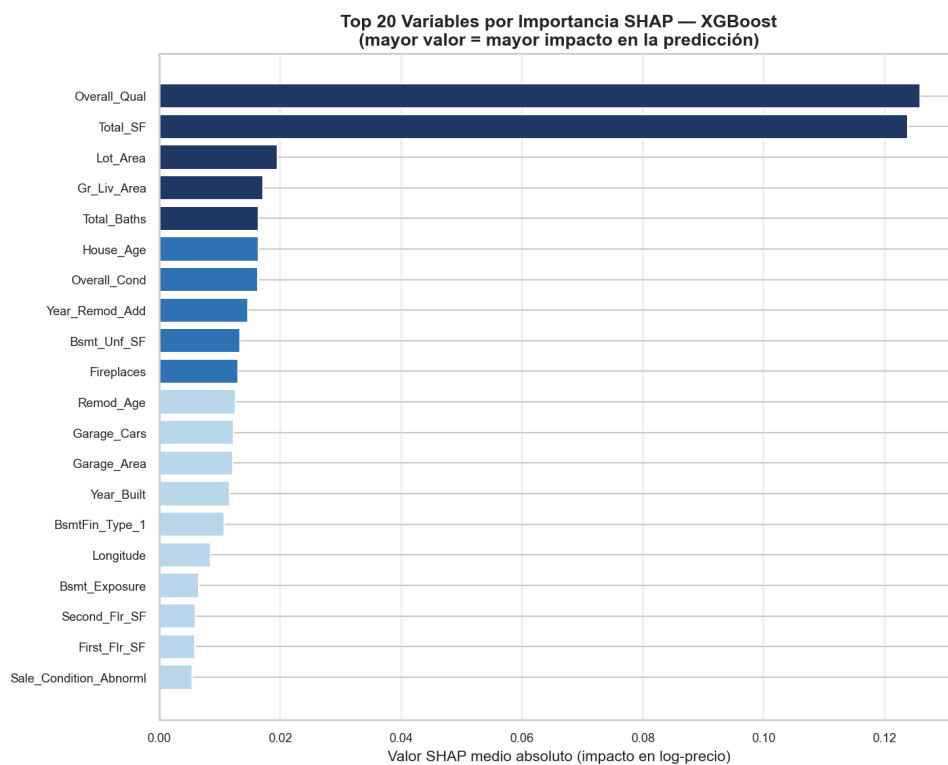


Figura 14: Importancia global de variables según valores SHAP sobre XGBoost.

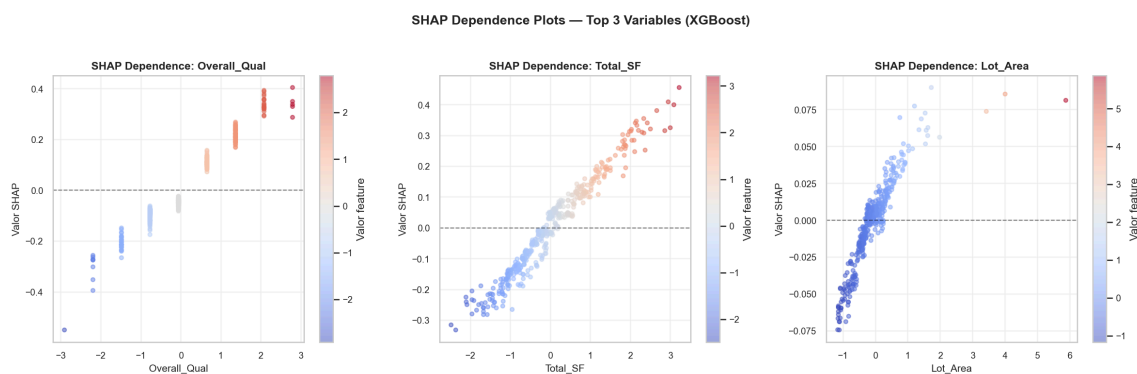


Figura 15: Gráficos de dependencia SHAP: revelan si la relación de cada variable con el precio es lineal o presenta umbrales.

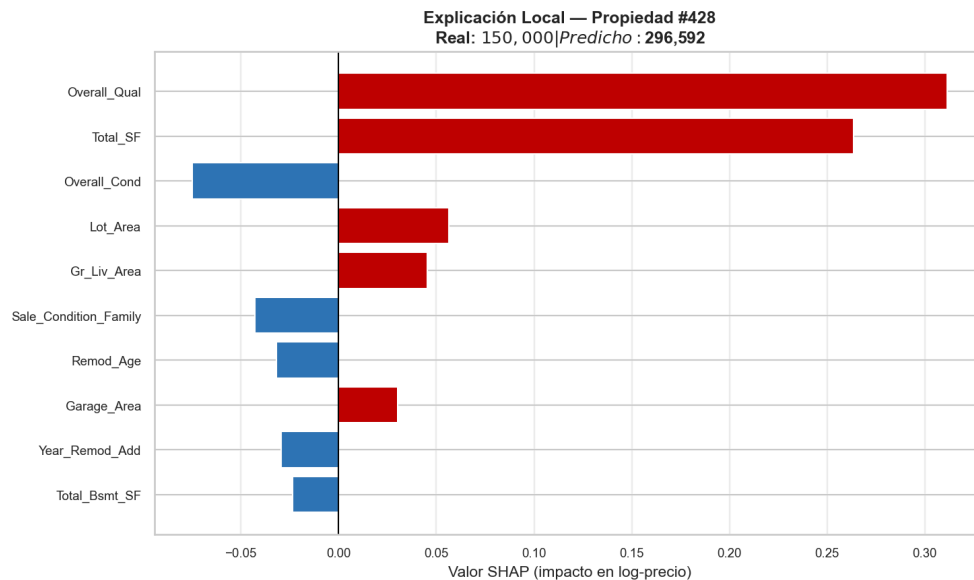


Figura 16: Descomposición SHAP de la **peor predicción** (propiedad #428: precio real \$150.000, predicho \$296.592). Permite explicar exactamente **por qué** el modelo se equivocó.

5.5 Análisis de Error por Segmento de Precio

El error no es uniforme. El segmento económico (< \ \$120k) concentra el mayor error relativo, por mayor heterogeneidad y menor densidad de datos.

Segmento	Ridge	XGBoost	AutoML
< \$120k (económicas)	11,85 %	10,41 %	12,63 %
\$120k – \$200k (medias)	7,84 %	6,99 %	7,16 %
\$200k – \$300k (premium)	6,65 %	8,03 %	7,81 %
> \$300k (lujo)	5,68 %	7,14 %	5,85 %

Error medio porcentual (MAPE) por segmento.

5.6 Conclusión Técnica: ¿Cuándo Conviene Deep Learning?

- **Hallazgo central:** en datos **tabulares y de volumen reducido** (~2.900 filas), los modelos clásicos regularizados (Ridge) **superan** a las redes profundas, son más interpretables y más baratos de entrenar. El Deep Learning **convendría** con datasets mucho mayores, interacciones fuertemente no lineales, o —como aquí— cuando se necesitan **embeddings** para la recuperación semántica. Ese es el valor real que el MLP aporta a la Fase 4.

6 DEPLOYMENT — MLOPS + RAG (FASE CRISP-DM 6)

6.1 Tracking de Experimentos con MLflow

Los **5 modelos** se registran en MLflow con hiperparámetros, métricas y artefactos (backend SQLite).

El mejor (Ridge) se promovió a **Production** en el Model Registry (ames_price_predictor v2).

Modelo	R ²	RMSE	Run ID
Ridge_Baseline	0,9359	\$20.035	0a021ca9...
AutoML_FLAML	0,9311	\$20.773	411a95c2...

Modelo	R ²	RMSE	Run ID
XGBoost	0,9219	\$22.106	a6da606f...
Random_Forest	0,9017	\$24.805	09a3b0ec...
MLP_Tabular	0,8619	\$29.401	e444902b...

6.2 Recuperación Semántica — RAG Tabular con FAISS

Los **embeddings 128D** del MLP se indexan con **FAISS** (IndexFlatIP sobre vectores L2-normalizados = similitud coseno). Dada una vivienda, el sistema recupera las más parecidas, ofreciendo **comparables** que contextualizan la tasación. Se considera **relevante** un vecino del mismo segmento de precio. Sobre 100 consultas aleatorias:

Segmento de la consulta	Precisión@5
Global	70,4 %
> \$300k	85,0 %
\$120k – \$200k	80,4 %
< \$120k	60,0 %
\$200k – \$300k	56,0 %

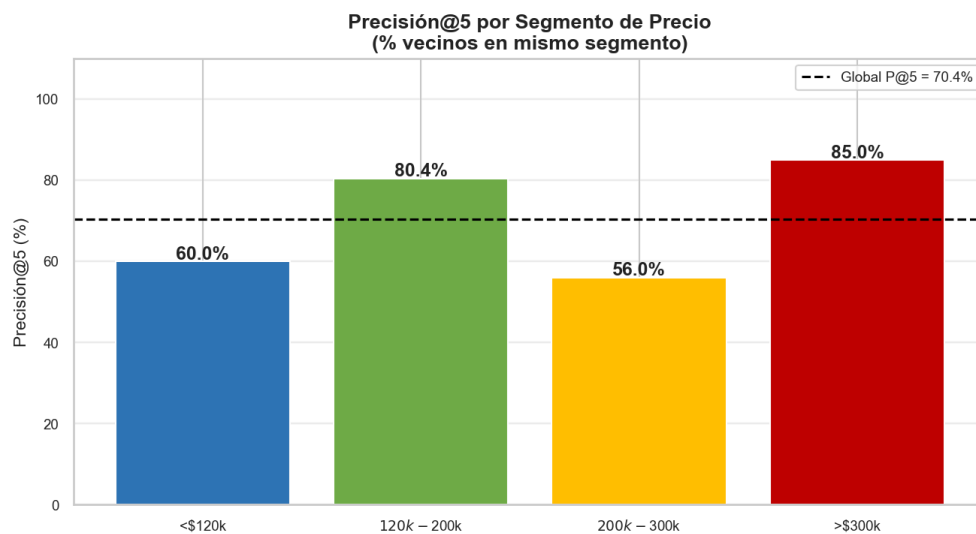


Figura 17: Precisión@5 por segmento. La recuperación es más fiable en los extremos del mercado (lujo y gama media) que en los segmentos de transición.

RAG Tabular — Propiedades Similares a #100 (\$280,000)

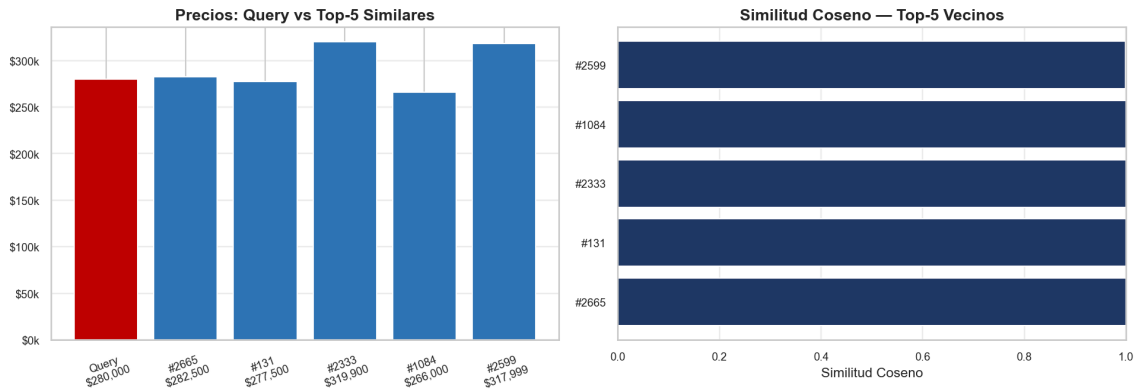


Figura 18: Ejemplo de recuperación: vivienda consulta (\$280.000) y sus 5 propiedades más similares con su similitud coseno (todas con similitud $\geq 0,999$).

6.2.1 Rendimiento en Producción: FAISS vs NearestNeighbors

Para validar la viabilidad en producción se comparó FAISS contra la búsqueda exacta de NearestNeighbors de scikit-learn sobre 200 consultas. FAISS es **196 veces más rápido**, manteniendo resultados idénticos:

Método	Tiempo / consulta	Total (200 q)
FAISS IndexFlatIP	0,09 ms	18,5 ms
NearestNeighbors (sklearn)	18,14 ms	3.627 ms
Speedup FAISS	196,4×	—

6.3 Monitoreo y Política de Reentrenamiento

Se monitorea el error por segmento de precio para detectar dónde se degrada el servicio (el segmento $< \$120k$ requiere atención prioritaria, según el análisis de error de la §5.3).

Dashboard de Monitoreo — Error por Segmento de Precio

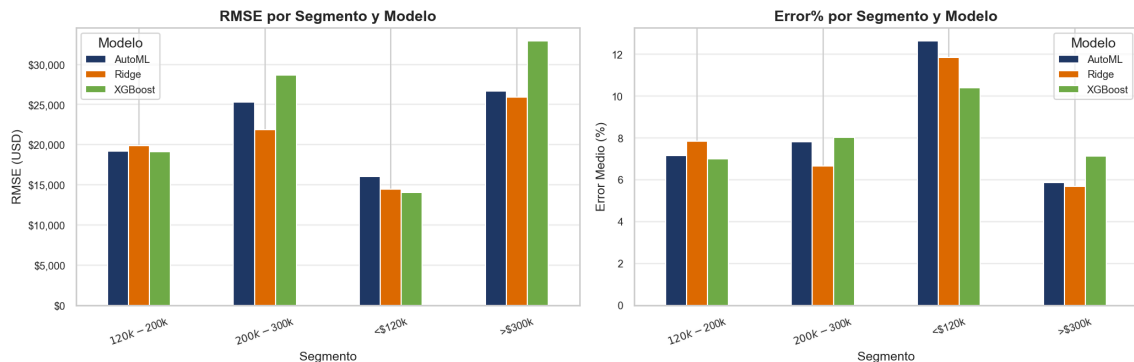


Figura 19: Dashboard de monitoreo: RMSE y error porcentual por segmento y modelo.

Se implementó el **Population Stability Index (PSI)** como disparador automático de reentrenamiento, calculado sobre las 263 features y validado con dos escenarios:

Escenario	PSI máximo	Acción
Datos de prueba normales (sin deriva)	$\approx 0,03$	No reentrenar ✓
Datos con deriva simulada (+2,5 σ)	8,46	Reentrenar modelo ✗

En el escenario sin deriva, todas las features quedaron por debajo de 0,10 (el sistema confirma estabilidad). En el escenario con deriva, features como Year_Remod_Add dispararon un **PSI = 8,46** (> 0,25), activando correctamente el trigger de reentrenamiento.

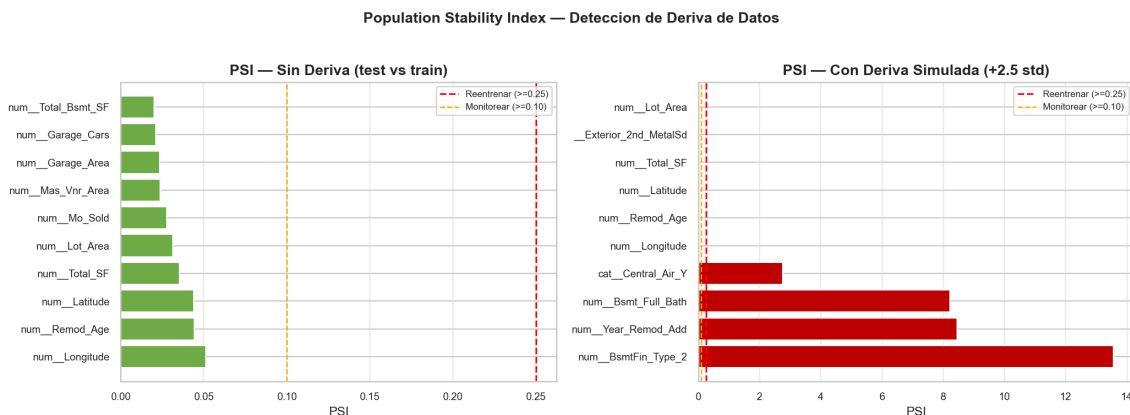


Figura 20: PSI por feature en ambos escenarios. El umbral $PSI \geq 0,25$ (línea roja) dispara el reentrenamiento cuando la distribución de producción se aleja de la de entrenamiento.

6.4 Demostración Funcional

- **Aplicación web (Streamlit):** el usuario ingresa atributos y obtiene el precio estimado más las 5 propiedades similares (app/streamlit_app.py).
- **CLI reproducible:** `python app/demo.py --idx 10 --k 5 --model ridge_baseline`.

— **Ejemplo de inferencia end-to-end:** Vivienda #10 → Precio real \$97.900 · **Precio predicho \$103.417** (error 5,64 %) · 5 comparables recuperadas en el mismo rango de precio.

7 INTEGRACIÓN FINAL Y CONTINUIDAD METODOLÓGICA

El proyecto no son tres ejercicios aislados, sino una **solución progresiva** con un hilo conductor explícito: los **embeddings** generados por el MLP en la Fase 3 son exactamente la materia prima que alimenta la **recuperación con FAISS** en la Fase 4.

Fase CRISP-DM	Qué se hizo	Evidencia
Business Understanding	Problema, objetivo, criterios de éxito	§1
Data Understanding	EDA, correlaciones, riesgos de calidad	01_EDA.ipynb
Data Preparation	Limpieza, encoding, split estratificado	02_Preprocessing.ipynb
Modeling	3 modelos + AutoML + K-Means + MLP/AE	03, 04, 05
Evaluation	Comparación, SHAP, error por segmento	03, 05
Deployment	MLflow, FAISS, PSI, demo	06_ML0ps.ipynb

8 CONCLUSIONES, LIMITACIONES Y TRABAJO FUTURO

8.1 Conclusiones

- Se cumplió el objetivo: un sistema de tasación con $R^2 = 0,9359$ y **RMSE = \$20.035**, superando todos los criterios de éxito definidos.

- **El modelo más simple ganó:** Ridge superó a XGBoost, Random Forest, al MLP profundo e incluso a AutoML, demostrando que la complejidad no garantiza mejor desempeño en datos tabulares.
- Los **embeddings** del Deep Learning encontraron su valor real no en la predicción, sino en la **recuperación semántica** (Precisión@5 = 70,4 %).
- El ciclo **MLOps** está operativo de extremo a extremo: tracking, registro, versionado, monitoreo y disparador de reentrenamiento.

8.2 Limitaciones

- **Volumen de datos reducido** (~2.900 filas) que penaliza a los modelos profundos.
- **Ausencia de variables textuales** y de **coordenadas geoespaciales** explícitas.
- El dataset corresponde a una sola ciudad y un periodo acotado (2006–2010): la **generalización** a otros mercados no está garantizada.
- El segmento económico (< \\$120k) mantiene el mayor error y la menor precisión de recuperación.

8.3 Trabajo Futuro

- Incorporar **descripciones textuales** y explotar Transformers preentrenados para embeddings de texto.
- Añadir **datos geoespaciales** (coordenadas, distancia a servicios) para capturar el efecto ubicación.
- Desplegar el servicio con **reentrenamiento automático** gatillado por el PSI en producción.
- Explorar **modelos calibrados por segmento** para reducir el error en el rango económico.

9 REFLEXIÓN CRÍTICA Y ÉTICA

9.1 Reflexión Crítica Metodológica

Más allá de los resultados, el proyecto deja lecciones metodológicas que conviene explicitar con honestidad:

- **La complejidad no garantiza desempeño.** El modelo más simple (Ridge) superó a XGBoost, Random Forest, al MLP profundo y a AutoML. La decisión defendible no fue «usar el modelo más potente», sino **elegir el modelo apropiado** al tamaño y naturaleza tabular del dataset.
- **AutoML como validación, no como atajo.** FLAML se usó para contrastar el diseño manual, no para reemplazar el razonamiento. Su resultado ($R^2 = 0,9311$) confirmó que el pipeline manual estaba bien planteado, evitando una «caja negra» innecesaria en producción.
- **El valor del Deep Learning fue indirecto.** El MLP no ganó en predicción, pero sus **embeddings** habilitaron la recuperación de similares (Fase 4). Reconocerlo evita sobrevender la red profunda como si fuera el mejor predictor.
- **Honestidad sobre los errores.** El modelo falla más en el segmento económico (< \\$120k, error $\approx 12\%$) y en casos atípicos (la peor predicción erró \$146.592). No se ocultan: se documentan y se proponen mitigaciones.

9.2 Sesgos y Equidad

- **Sesgo geográfico (el más relevante).** El precio correlaciona fuertemente con el barrio (Neighborhood), y el clustering confirmó coherencia geográfica entre segmentos y zonas. Un modelo de tasación entrenado así puede **perpetuar o amplificar desigualdades**: si una zona está históricamente subvalorada, el modelo aprenderá y reforzará esa subvaloración.

- **Sesgo temporal.** Los datos cubren 2006–2010, incluyendo la crisis hipotecaria de 2008. Las estimaciones reflejan ese contexto y **no deben extrapolarse** a mercados actuales sin recalibrar.
- **Sesgo de representación.** Los segmentos de lujo y económico tienen menos datos, lo que produce mayor error en esos rangos: el modelo es más justo con la vivienda «promedio».

9.3 Transparencia e Interpretabilidad

La interpretabilidad fue una decisión de diseño, no un añadido. La elección de Ridge (lineal) y el uso de **SHAP** permiten **explicar cada predicción individual**: qué variables la subieron o bajaron y en qué magnitud. Esto hace el sistema **auditable** y permite a un tasador humano detectar cuándo el modelo razona de forma incorrecta.

9.4 Uso Responsable

- La herramienta es un **apoyo a la decisión**, nunca un sustituto del juicio profesional ni una autoridad final sobre el valor de una vivienda.
- Una estimación automática **no debe usarse para negar, encarecer o condicionar** el acceso a la vivienda de forma automatizada.
- El sistema entrega un **rango** y propiedades comparables justamente para que el usuario contextualice y cuestione la cifra, en lugar de aceptarla ciegamente.

9.5 Privacidad de Datos

El dataset Ames Housing es **público, anonimizado y no contiene información personal identificable** (PII) de compradores o vendedores. No existe riesgo de privacidad sobre individuos, y su procedencia es plenamente trazable (OpenML ID 41211).

9.6 Honestidad Académica

Todos los resultados de este informe provienen de la **ejecución real del código** del repositorio (no son cifras estimadas ni inventadas). Las decisiones de muestreo, descarte de outliers y transformaciones quedaron registradas; las librerías, datasets y herramientas utilizadas se declaran explícitamente en la sección de Referencias.

- **Síntesis ética:** un sistema de tasación es técnicamente útil pero socialmente sensible. Su valor responsable depende de tres condiciones: que sea **transparente** (SHAP), que se use como **apoyo y no como autoridad**, y que se **monitoree** (PSI) para no operar sobre datos que ya no representan la realidad.

10 REFERENCIAS

-
- [1] De Cock, D. (2011). *Ames, Iowa: Alternative to the Boston Housing Data Set*. Journal of Statistics Education, 19(3). Dataset: OpenML ID 41211.
 - [2] Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. JMLR 12.
 - [3] Chen, T. & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. KDD ‘16.
 - [4] Wang, C. et al. (2021). *FLAML: A Fast and Lightweight AutoML Library*. MLSys.
 - [5] Paszke, A. et al. (2019). *PyTorch: High-Performance Deep Learning Library*. NeurIPS.
 - [6] Johnson, J., Douze, M. & Jégou, H. (2019). *Billion-scale similarity search with GPUs (FAISS)*. IEEE Big Data.

- [7] Lundberg, S. & Lee, S. (2017). *A Unified Approach to Interpreting Model Predictions (SHAP)*. NeurIPS.
- [8] Zaharia, M. et al. (2018). *Accelerating the ML Lifecycle with MLflow*.
- [9] Wirth, R. & Hipp, J. (2000). *CRISP-DM: Towards a Standard Process Model for Data Mining*.
- [10] Herramientas: Python 3.11, scikit-learn, XGBoost, FLAML, PyTorch, MLflow, FAISS, SHAP, Streamlit, Typst.

11 ANEXOS

11.1 Anexo A — Estructura del Repositorio

Proyecto-Integrador/

```
├─ data/
│   ├── raw/          ames_housing_raw.csv          (dataset crudo OpenML)
│   └─ processed/     X_{train,val,test}.npz · y_{train,val,test}.npz
│                       train/val/test.csv · feature_names.json
│                       metadata.json                (trazabilidad + hashes)
├─ notebooks/
│   ├── 01_EDA.ipynb      Fase 2 – Data Understanding
│   ├── 02_Preprocessing.ipynb Fase 2 – Data Prep + Feature Eng.
│   ├── 03_ML_Models.ipynb Fase 2 – ML + AutoML + SHAP
│   ├── 04_Clustering.ipynb Fase 2 – K-Means (no supervisado)
│   ├── 05_DeepLearning.ipynb Fase 3 – MLP + Autoencoder
│   └─ 06_MLOps.ipynb     Fase 4 – MLflow + FAISS + PSI
├─ src/
│   ├── data_loader.py    preprocessing.py    models.py
│   ├── clustering.py     deep_learning.py    similarity.py
│   ├── mlops.py          predict.py          utils.py
├─ models/
│   ├── ridge_baseline.pkl · random_forest.pkl · xgboost.pkl
│   ├── automl_flaml.pkl · kmeans.pkl · cluster_scaler.pkl
│   ├── preprocessor.pkl · flaml_log.json
│   ├── mlp_tabular.pt · autoencoder.pt
│   └─ mlp_embeddings.npy · ae_embeddings.npy · *_labels.npy
├─ reports/
│   ├── figures/          33 figuras (.png)
│   ├── model_comparison.csv
│   ├── informe_final.typ + informe_final.pdf
├─ app/
│   ├── streamlit_app.py (demo web)  demo.py (demo CLI)
├─ config/
│   └─ params.yaml        (hiperparámetros y configuración)
├─ .streamlit/config.toml  mlflow.db    (tracking MLflow)
├─ requirements.txt        README.md     PRODUCT.md
```

11.2 Anexo B — Configuración del AutoML (FLAML)

Mejor estimador: `xgb_limitdepth`. Configuración: `n_estimators = 1.291`, `max_depth = 2`, `learning_rate = 0,0595`, `subsample = 0,594`, `colsample_bytree = 0,813`, `reg_alpha = 0,032`, `reg_lambda = 0,570`.

Presupuesto: 300 s sobre la misma partición.

11.3 Anexo C — Instrucciones de Ejecución

1. Instalar dependencias

```
pip install -r requirements.txt
```

2. Ejecutar notebooks en orden (01 → 06)

3. Lanzar la demo web

```
streamlit run app/streamlit_app.py
```

4. Inferencia por CLI

```
python app/demo.py --idx 10 --k 5 --model ridge_baseline
```

5. Ver experimentos MLflow

```
mlflow ui # http://localhost:5000
```

11.4 Anexo D — Reproducibilidad

- Semilla global: `random_state = 42` en todos los scripts y notebooks.
- Partición: 70 / 15 / 15 estratificada por quintiles de precio.
- Pipeline ajustado solo en `train`; aplicado a `val` y `test`.
- Hashes MD5 de cada artefacto registrados en `data/processed/metadata.json`.